

Enterprise RAG AI Document Search — Complete PRD for Cursor

Everything You Need to Build This Project from Scratch

Version: 1.0 | **Date:** June 2026

Stack: Python 3.11 + FastAPI + LangChain + ChromaDB + Next.js 14 + Tailwind + shadcn/ui

HOW TO USE THIS PRD IN CURSOR

1. Open Cursor → create a new empty project folder called `enterprise-rag`
 2. Open Cursor Chat (`Cmd+L`) / (`Ctrl+L`) → paste the **exact prompt from Section 1** first
 3. Then use the **per-phase prompts** in Sections 2-6 one at a time
 4. Each prompt tells Cursor exactly what to build — paste, run, verify, move to next
-

SECTION 1 — MASTER CONTEXT PROMPT (Paste This First)

Paste this once at the start of your Cursor session so it understands the full project:

I am building an Enterprise AI Document Search & Question Answering System using RAG (Retrieval-Augmented Generation). Here is the complete context:

PROJECT NAME: Enterprise RAG AI Assistant

GOAL: Allow employees to ask natural language questions against uploaded company documents (PDFs, DOCX, TXT) and get grounded answers with source citations. Zero hallucination – if the answer is not in documents, the system refuses to answer.

TECH STACK:

- Backend: Python 3.11, FastAPI, LangChain, ChromaDB (vector store), SQLite (metadata), OpenAI gpt-4o-mini + text-embedding-3-small
- Frontend: Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui
- Containerization: Docker + docker-compose

MONOREPO STRUCTURE:

```
enterprise-rag/
├── backend/
│   ├── main.py
│   ├── config.py
│   ├── database.py
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── ingestion.py
│   │   ├── retriever.py
│   │   └── generator.py
│   ├── documents/           (runtime upload temp folder)
│   ├── vector_db/           (ChromaDB persistence)
│   ├── database/            (SQLite file)
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   └── src/
```

```
| | | └─ app/
| | | | └─ layout.tsx
| | | | └─ page.tsx
| | | | └─ globals.css
| | | └─ components/
| | | | └─ chat/
| | | | | └─ ChatContainer.tsx
| | | | | └─ MessageBubble.tsx
| | | | | └─ ChatInput.tsx
| | | | └─ admin/
| | | | | └─ UploadModal.tsx
| | | | | └─ DocManager.tsx
| | | | | └─ AnalyticsView.tsx
| | └─ package.json
| | └─ Dockerfile
└─ docker-compose.yml
└─ .env.example
```

TWO PORTALS:

1. Employee Portal (Chat Tab): Type questions, get answers with source citations and confidence scores
2. Admin Portal (Admin Hub Tab): Upload documents, manage indexed files, view analytics dashboard

KEY BUSINESS RULES:

- Chunk size: 1000 chars, overlap: 200 chars
- Hybrid search: 75% vector (ChromaDB cosine) + 25% BM25 keyword
- Confidence: $\geq 85\%$ = Green, 70-84% = Yellow, $< 70\%$ = Red / refuse
- If no relevant context found $\rightarrow$ return "I couldn't find this information in the uploaded documents."
- Every answer must cite: filename, page number, relevance score
- Supported file types: PDF, DOCX, TXT

DATABASE TABLES:

- documents (id, filename, file_type, file_size, upload_timestamp, chunk_count, status)
- search_analytics (id, query_text, response_time_ms, confidence_score, top_document_matched, timestamp)
- chat_sessions (session_id, user_id, created_at)
- chat_messages (id, session_id, role, content, citations_json, timestamp)

API ENDPOINTS:

- POST /api/v1/documents/upload → Upload & index file
- GET /api/v1/documents → List all indexed documents
- DELETE /api/v1/documents/{id} → Delete document + purge vectors
- POST /api/v1/chat/query → Ask a question, get grounded answer
- GET /api/v1/analytics/stats → Dashboard metrics

UI COLOR THEME:

- Background: #0b0f19 (deep navy)
- Card surface: #1e293b (slate)
- Accent: #6366f1 (indigo)
- Success: #10b981 (emerald)
- Warning: #f59e0b (amber)
- Font: Inter (UI), JetBrains Mono (code/citations)

I will give you instructions phase by phase. Wait for my next message.

SECTION 2 — PHASE 1: Backend Foundation

Paste this as your next Cursor prompt:

Phase 1: Set up the complete FastAPI backend.

Create the following files inside the `backend/` folder:

```
### backend/requirements.txt
```

```
fastapi==0.111.0 uvicorn==0.30.1 pydantic==2.7.4 pydantic-settings==2.3.4  
langchain==0.2.5 langchain-community==0.2.5 langchain-openai==0.1.9  
chromadb==0.5.3 pypdf==4.2.0 python-docx==1.1.2 rank_bm25==0.2.2 python-  
multipart==0.0.9 aiofiles==23.2.1
```

```

### backend/config.py
```python
import os
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
 APP_NAME: str = "Enterprise RAG AI Assistant"
 OPENAI_API_KEY: str = os.getenv("OPENAI_API_KEY", "sk-placeholder")
 CHROMA_PERSIST_DIR: str = "./vector_db"
 SQLITE_DB_PATH: str = "./database/enterprise_rag.db"
 EMBEDDING_MODEL: str = "text-embedding-3-small"
 LLM_MODEL: str = "gpt-4o-mini"
 CHUNK_SIZE: int = 1000
 CHUNK_OVERLAP: int = 200
 TOP_K_RESULTS: int = 4

settings = Settings()
```

```

```

### backend/database.py
Create a SQLite connection manager with these functions:
- `init_db()` → creates all 4 tables (documents, search_analytics,
chat_sessions, chat_messages)
- `get_db_connection()` → returns sqlite3 connection with row_factory =
sqlite3.Row

```

Tables DDL:

```

```sql
CREATE TABLE IF NOT EXISTS documents (
 id TEXT PRIMARY KEY,
 filename TEXT NOT NULL,
 file_type TEXT NOT NULL,

```

```
file_size INTEGER NOT NULL,
upload_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
chunk_count INTEGER DEFAULT 0,
status TEXT CHECK(status IN ('INDEXING','COMPLETED','FAILED'))
);
```

```
CREATE TABLE IF NOT EXISTS search_analytics (
id INTEGER PRIMARY KEY AUTOINCREMENT,
query_text TEXT NOT NULL,
response_time_ms INTEGER NOT NULL,
confidence_score REAL NOT NULL,
top_document_matched TEXT,
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS chat_sessions (
session_id TEXT PRIMARY KEY,
user_id TEXT NOT NULL,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS chat_messages (
id INTEGER PRIMARY KEY AUTOINCREMENT,
session_id TEXT REFERENCES chat_sessions(session_id),
role TEXT CHECK(role IN ('user','assistant')),
content TEXT NOT NULL,
citations_json TEXT,
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
...
```

```
backend/rag/__init__.py
Empty file.
```

```
backend/rag/ingestion.py
```

```
Build a `DocumentIngestionEngine` class with:
```

- `\_\_init\_\_`: sets up OpenAIEmbeddings (text-embedding-3-small), Chroma vector store (collection: "enterprise\_docs"), and RecursiveCharacterTextSplitter (size=1000, overlap=200, separators=["\n\n","\n"," ",""])
- `process\_and\_index\_file(file\_path, original\_filename) -> dict`:
  1. Detects file type (pdf/docx/txt) and uses correct LangChain loader (PyPDFLoader, Docx2txtLoader, TextLoader)
  2. Splits into chunks
  3. Adds metadata: document\_id (uuid4), filename, page\_number, chunk\_index to each chunk
  4. Calls vector\_store.add\_documents(chunks)
  5. Inserts record into SQLite documents table with status="COMPLETED"
  6. Returns {document\_id, chunks\_indexed, status}

```
backend/rag/retriever.py
```

```
Build a `HybridRetriever` class:
```

- `\_\_init\_\_`: sets up OpenAIEmbeddings + Chroma vector store (same collection)
- `retrieve(query, top\_k=4) -> list`:
  1. Calls similarity\_search\_with\_relevance\_scores(query, k=top\_k)
  2. Returns list of {content, filename, page, relevance\_score}

```
backend/rag/generator.py
```

```
Build a `RAGGenerator` class:
```

- `\_\_init\_\_`: ChatOpenAI (gpt-4o-mini, temperature=0.0) + HybridRetriever
- `generate\_answer(query, session\_id) -> dict`:
  1. Retrieves top\_k chunks
  2. If none found → return refusal response immediately
  3. Builds context string with [Source N: filename - Page X] labels
  4. Calculates confidence = avg of relevance scores, clamped 0-1



```
5. System prompt enforces: answer ONLY from context, zero hallucination,
cite sources
6. Invokes LLM
7. If "couldn't find" in answer → set confidence=0.0
8. Logs to search_analytics table
9. Returns {answer, confidence_score (0-100), sources, response_time_ms}
```

Anti-hallucination system prompt template:

You are an expert Enterprise Document Assistant. Answer ONLY using the retrieved context below.

**RULES:**

1. If the answer is not in the context, reply EXACTLY: "I couldn't find this information in the uploaded documents."
2. Do not fabricate facts or use outside knowledge.
3. Keep answers precise and professional.
4. Every claim must be traceable to the provided sources.

=== RETRIEVED CONTEXT === {context\_str}

```
backend/main.py
FastAPI app with:
- CORS middleware (allow_origins=["*"])
- startup event → database.init_db()
- Pydantic models: QueryRequest (session_id, query), QueryResponse (answer,
confidence_score, sources, response_time_ms)
- All 5 API routes implemented:
 - POST /api/v1/documents/upload (saves to ./documents/temp, calls
ingestion engine, deletes temp file)
 - GET /api/v1/documents (returns all from SQLite, ordered by
upload_timestamp DESC)
 - DELETE /api/v1/documents/{doc_id} (deletes from SQLite + purges from
ChromaDB by metadata filter)
 - POST /api/v1/chat/query (calls RAGGenerator.generate_answer)
 - GET /api/v1/analytics/stats (returns total_queries,
average_response_time_ms, average_confidence_score, popular_documents top
5)
- Main guard: uvicorn.run("main:app", host="0.0.0.0", port=8000,
reload=True)
```

```
backend/Dockerfile
```dockerfile
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y --no-install-recommends gcc build-
essential libsqlite3-dev && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
...
```

After creating all files, verify by running:

```
cd backend && pip install -r requirements.txt && python main.py
```

The server should start at <http://localhost:8000>. Check

<http://localhost:8000/docs> for the auto-generated Swagger UI.

SECTION 3 — PHASE 2: Next.js Frontend Setup

Phase 2: Set up the Next.js 14 frontend.

Run these commands first:

```
```bash
cd enterprise-rag
npx create-next-app@14 frontend --typescript --tailwind --eslint --app --
src-dir --import-alias "@/*"
cd frontend
npx shadcn-ui@latest init
npx shadcn-ui@latest add button badge tabs card input textarea dialog
progress
npm install lucide-react
```
```

Then create a `.env.local` file in the `frontend/` folder:

`NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1`

Update `frontend/src/app/globals.css` to add these CSS variables and base styles:

```
```css
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
 --bg-primary: #0b0f19;
 --bg-card: #1e293b;
 --accent: #6366f1;
 --success: #10b981;
 --warning: #f59e0b;
 --danger: #ef4444;
}

body {
 background-color: #0b0f19;
 color: #e2e8f0;
 font-family: 'Inter', sans-serif;
}
```
```

Update `frontend/tailwind.config.ts` to extend colors:

```
```ts
colors: {
 navy: { DEFAULT: '#0b0f19', card: '#1e293b' },
 indigo: { DEFAULT: '#6366f1' },
 emerald: { DEFAULT: '#10b981' },
 amber: { DEFAULT: '#f59e0b' },
}
```
```

Create a shared API client at `frontend/src/lib/api.ts`:

```
``ts
const BASE = process.env.NEXT_PUBLIC_API_URL;

export async function queryRAG(query: string, sessionId: string) {
  const res = await fetch(`${BASE}/chat/query`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ query, session_id: sessionId }),
  });
  if (!res.ok) throw new Error('Query failed');
  return res.json();
}

export async function uploadDocument(file: File) {
  const form = new FormData();
  form.append('file', file);
  const res = await fetch(`${BASE}/documents/upload`, { method: 'POST',
body: form });
  if (!res.ok) throw new Error('Upload failed');
  return res.json();
}

export async function getDocuments() {
  const res = await fetch(`${BASE}/documents`);
  return res.json();
}

export async function deleteDocument(id: string) {
  const res = await fetch(`${BASE}/documents/${id}`, { method: 'DELETE' });
  return res.json();
}
```

```
export async function getAnalytics() {
  const res = await fetch(`${BASE}/analytics/stats`);
  return res.json();
}
...
```

SECTION 4 — PHASE 3: Chat Components

Phase 3: Build the Employee Chat Portal components.

frontend/src/components/chat/ChatInput.tsx

A textarea input with:

- Auto-resize on typing (up to 5 rows max)
- Send button (indigo) + keyboard shortcut: Enter to send, Shift+Enter for newline
- Disabled state when `isLoading` prop is true (show spinner in button)
- Placeholder: "Ask a question about your company documents..."
- Props: { onSend: (msg: string) => void, isLoading: boolean }
- Dark styling: bg-[#1e293b] border border-slate-700 rounded-xl text-slate-200

frontend/src/components/chat/MessageBubble.tsx

Renders a single chat message. Props: { role: 'user'|'assistant', content: string, sources?: Source[], confidenceScore?: number, responseTimeMs?: number }

Where Source = { filename: string, page: number, excerpt: string, relevance_score: number }

For user messages: right-aligned bubble with indigo-600 background

For assistant messages:

- Left-aligned with bg-[#1e293b] background
- Render the answer text in white
- Below the answer, if sources exist, render a "Sources" section:
 - A confidence badge: green if ≥ 85 , yellow if ≥ 70 , red if < 70 (show percentage)
 - Small gray "Response: {responseTimeMs}ms" text
 - For each source: a pill/card showing 📄 {filename} • Page {page} • {relevance_score*100}% match
 - Clicking a source pill opens a CitationModal showing the full excerpt text

frontend/src/components/chat/CitationModal.tsx

A shadcn Dialog that shows:

- Title: "{filename} – Page {page}"
- Body: The full `excerpt` text in a scrollable monospace (JetBrains Mono) block
- Relevance score as a progress bar
- Close button

frontend/src/components/chat/ChatContainer.tsx

Main chat viewport. Props: { sessionId: string }

State: messages (array of message objects), isLoading (bool)

Behavior:

- Renders list of MessageBubble components
- Auto-scrolls to bottom on new messages (useRef + useEffect)
- On user send: immediately adds user message, calls queryRAG(), appends assistant message
- While loading: show a typing indicator (3 animated dots) as a placeholder assistant bubble

- Empty state: centered illustration + text "Ask anything about your company documents"
- Full height scrollable area with overflow-y-auto

SECTION 5 — PHASE 4: Admin Components

Phase 4: Build the Admin Hub components.

frontend/src/components/admin/UploadModal.tsx

A shadcn Dialog triggered by an "Upload Document" button.

Inside the dialog:

- Drag-and-drop zone: dashed border, on hover highlight with indigo tint
 - Supports PDF, DOCX, TXT only (validate on client before upload)
 - After file selected, show file name + size
 - "Upload & Index" button: calls uploadDocument() from api.ts
 - During upload: show a progress bar (animate from 0→100 over estimated time, or spinner)
 - On success: show green checkmark + "Document indexed successfully!"
- Chunks: {n}"
- On error: show red error message
 - After success, emit onSuccess callback to trigger document list refresh

frontend/src/components/admin/DocManager.tsx

A card showing all indexed documents as a table.

Columns: Filename | Type Badge | Size | Chunks | Uploaded | Status |
Actions

- Fetches documents on mount using getDocuments()
- Each row has a red "Delete" button that calls deleteDocument(id) with a


```
confirmation dialog before deleting
- Status column: "COMPLETED" = green badge, "INDEXING" = yellow badge +
spinner, "FAILED" = red badge
- File type badges: PDF = red, DOCX = blue, TXT = gray
- Empty state: "No documents indexed yet. Upload your first document
above."
- Refresh button in card header
```

```
### frontend/src/components/admin/AnalyticsView.tsx
A dashboard card showing stats fetched from getAnalytics().
```

Display 3 stat tiles in a grid:

1. Total Queries (with a search icon)
2. Avg Response Time in ms (with a clock icon)
3. Avg Confidence Score as % (with a chart icon)

Below the tiles, show a "Most Searched Documents" list:

- Ranked list (1-5) of top_document_matched with query count
- Each item shows the document name + a horizontal bar showing relative popularity

Auto-refresh every 30 seconds.

SECTION 6 — PHASE 5: Main Page & Layout

```
Phase 5: Wire everything together in the main page.
```

```
### frontend/src/app/layout.tsx
Standard Next.js layout with:
- Inter font from Google Fonts
- Background: bg-[#0b0f19] min-h-screen
```

- Metadata: title="Enterprise AI Document Search", description="RAG-powered corporate knowledge base"

frontend/src/app/page.tsx

The main application page. This is a TWO-TAB layout.

Header bar (sticky top):

- Left: App logo (a small indigo robot or sparkle icon) + "Enterprise AI" in bold white + "Document Search" in slate-400
- Right: Two tab buttons – "Employee Chat" and "Admin Hub" – styled like pills, active tab has indigo background

Tab 1 – Employee Chat:

- Full height split or single-column layout
- Renders ChatContainer with a unique sessionId (generate once with uuid on mount, store in useState)
- Header above chat: "Ask Your Documents" with subtitle "Powered by RAG – every answer is sourced and verified"

Tab 2 – Admin Hub:

- Two-column grid layout (on desktop):
 - Left column: UploadModal trigger button + DocManager table
 - Right column: AnalyticsView dashboard
- Header: "Admin Control Center" with subtitle "Manage your knowledge base"

State: activeTab ('chat' | 'admin'), sessionId

Both tabs should be rendered (not conditionally mounted) so state is preserved on tab switch. Use CSS show/hide.

The overall page should feel like a production enterprise SaaS app – dark navy background, card surfaces in slate-800, clean typography, generous padding.

SECTION 7 — PHASE 6: Docker & Final Setup

Phase 6: Add Docker configuration and project finalization.

```
### docker-compose.yml (in project root)
```yaml
version: '3.8'
services:
 backend:
 build: ./backend
 container_name: rag_backend
 restart: always
 ports:
 - "8000:8000"
 environment:
 - OPENAI_API_KEY=${OPENAI_API_KEY}
 volumes:
 - ./backend/vector_db:/app/vector_db
 - ./backend/database:/app/database
 - ./backend/documents:/app/documents

 frontend:
 image: node:20-alpine
 container_name: rag_frontend
 working_dir: /app
 volumes:
 - ./frontend:/app
 command: sh -c "npm install && npm run dev"
 ports:
 - "3000:3000"
 environment:
```

```
- NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
depends_on:
 - backend
...

.env.example (in project root)
```

OPENAI\_API\_KEY=sk-your-openai-key-here

```
README.md (in project root)
Generate a complete README with:
1. Project overview (1 paragraph)
2. Prerequisites: Python 3.11+, Node 20+, OpenAI API key, Docker (optional)
3. Quick Start (local dev):
```bash
# Backend
cd backend
pip install -r requirements.txt
cp ../.env.example .env # add your API key
python main.py          # starts at :8000

# Frontend (new terminal)
cd frontend
npm install
npm run dev              # starts at :3000
...
4. Docker start: `docker-compose up --build`
5. Environment variables table
6. API endpoints table (method, path, description)
7. Architecture diagram (copy ASCII diagram from project doc)
8. Tech stack badges

After all files are created:
1. Verify backend: `curl http://localhost:8000/docs` should show Swagger UI
2. Verify frontend: open http://localhost:3000 in browser
3. Test end-to-end: upload a PDF via Admin Hub → ask a question in Employee
Chat → verify answer has source citations
```

SECTION 8 — QUICK REFERENCE: What to Give Cursor

Here is the exact sequence of what to paste into Cursor, one at a time:

Step	What to Paste	What Gets Built
1	Section 1 — Master Context	Sets project understanding
2	Section 2 — Phase 1	Entire FastAPI backend (5 files)
3	Section 3 — Phase 2	Next.js setup + API client
4	Section 4 — Phase 3	Chat UI components
5	Section 5 — Phase 4	Admin UI components
6	Section 6 — Phase 5	Main page wiring
7	Section 7 — Phase 6	Docker + README

After each phase: Test what was built before moving to the next. Fix errors with Cursor inline — just highlight the error and press `Cmd+K`.

SECTION 9 — KEY DECISIONS TO CONFIRM BEFORE STARTING

Answer these before you begin, and tell Cursor your choices upfront:

Decision	Default (Recommended)	Alternative
Do you have an OpenAI API key?	Yes, use gpt-4o-mini	No → use Ollama + llama3 locally
Run locally or Docker?	Local dev first, Docker later	Docker from day 1
Authentication needed?	No auth (open access)	Add simple JWT or session tokens
Database	SQLite (zero setup)	PostgreSQL (more robust)
Frontend port	3000	Any open port
Backend port	8000	Any open port

SECTION 10 — ENVIRONMENT VARIABLES

Create `backend/.env` with:

```
OPENAI_API_KEY=sk-xxxxxxxxxxxxx
CHROMA_PERSIST_DIR=./vector_db
SQLITE_DB_PATH=./database/enterprise_rag.db
EMBEDDING_MODEL=text-embedding-3-small
LLM_MODEL=gpt-4o-mini
CHUNK_SIZE=1000
CHUNK_OVERLAP=200
TOP_K_RESULTS=4
```

Create `frontend/.env.local` with:

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
```

SECTION 11 — COMMON ERRORS & FIXES

Error	Cause	Fix
<code>ModuleNotFoundError: pydantic_settings</code>	Missing package	<code>pip install pydantic-settings</code>
<code>chromadb.errors.InvalidDimensionException</code>	Mixed embedding models	Delete <code>vector_db/</code> folder, restart
CORS error in browser	Backend CORS not configured	Ensure <code>CORSMiddleware</code> is in <code>main.py</code>
<code>404 on /api/v1/...</code>	Wrong <code>NEXT_PUBLIC_API_URL</code>	Check <code>.env.local</code> and restart <code>Next.js</code>
ChromaDB import error	Version mismatch	Use <code>chromadb==0.5.</code> exactly
<code>pypdf</code> not found	Old import style	Use <code>pypdf</code> not <code>PyPDF2</code> in requirements

Error	Cause	Fix
LangChain deprecation warnings	API changes in v0.2	Use <code>langchain-community</code> for loaders

This PRD was generated from the Enterprise RAG Master Project Document. Give Sections 1-7 to Cursor in sequence and your full-stack project will be production-ready.